Augmenting Snapshot-based Methods with Neighborhood Features

Jingyi Wan[*]
University of Cambridge
jw2428@cam.ac.uk

Tali Motzkin[*]
University of Cambridge
tm861@cam.ac.uk

Andy Huang
University of Cambridge
shenyang.huang@mail.mcgill.ca

Abstract

Temporal graph learning methods typically fall into two categories: snapshot-based models, which are efficient as they process data in aggregated time intervals, and event-based models, which often achieve higher accuracy by leveraging joint neighborhood features such as common neighbors. Motivated by this observation, we propose enhancing snapshot-based models, under a unified framework, with joint neighborhood features encoded as positional information, aiming to boost predictive performance while maintaining computational efficiency. Experiments results on `tgbl-review` and `tgbl-wiki` datasets have demonstrated the effectiveness of encoding joint neighborhood features which achieves better results compared to snapshot-based models.

Statement of contribution

We, Jingyi Wan and Tali Motzkin of the University of Cambridge, jointly declare that our work towards this project has been executed as follows:

- **Jingyi Wan** contributed by running DyGFormer-related experiments, documenting DyGFormer-related content, and implementing the proposed joint neighborhood feature encoding method with ablation studies.
- **Tali Motzkin** contributed by running NAT-related experiments, documenting NAT-related content, and implementing the proposed joint neighborhood feature encoding method with ablation studies.

We both independently submit identical copies of this paper, certifying this statement to be correct.

GitHub repository with commit log

The companion source code for our project may be found at: https://github.com/TaliMotzkin/UTG/tree/Main_Project and https://github.com/Jingyi-Official/AugSnapshot.git.

1 Introduction

Dynamic graph learning has recently become a trending research topic due to its effectiveness in various real-world scenarios, such as social networks, user-item interaction systems, traffic networks, and physical systems. Temporal graphs (TGs) are primarily categorized into two types [1]: Discrete Time Dynamic Graphs (DTDGs) and Continuous Time Dynamic Graphs (CTDGs). While both types of TGs are graphs that evolve over time, forming different structures as edges, nodes, and their features change [2], CTDGs are characterized by an edge stream that manifests the timeline. In contrast, DTDGs represent changes through a more coarse-grained timeline, with snapshot-based modifications.

[*]Equal contribution.

Jingyi Wan & Tali Motzkin & Andy Huang, Augmenting Snapshot-based Methods with Neighborhood Features. *Geometric Deep Learning (L65), University of Cambridge*.

Both types of representation can be beneficial for different tasks; however, research on these two types of graphs is mostly developed in isolation [3], not jointly benefiting from each other's advantages. For example, the snapshot-based approach is more practical, as it allows for an easier summary of node information [4], and holding less granular information leads to faster processing [3]. On the other hand, event-based graphs, which include fine-grained time information, can improve precision [5]. Moreover, when decision-makers face a choice between these two options–a decision that often boils down to design preferences–it becomes challenging to compare them directly [3].

To address these gaps, [3] implemented a framework called Unified Temporal Graph (UTG), which combines both types of TGs, benefiting from the advantages of each and enabling streamlined comparisons between the two designs and their datasets. Evaluating the task of link prediction, the UTG framework increased the performance of snapshot-based models. However, they found that event-based methods such as NAT and DyGFormer, which leverage information about the joint neighborhood between source and target nodes of a link, performed better. This finding leads to the main goal of our work. Our aim is to examine whether neighborhood structural information can further enhance the performance of link prediction, using a GLSTM model within the UTG framework and with CTDG datasets.

2 Background

2.1 Discrete Time Dynamic Graphs

DTDGs are defined as a sequence of complete graph snapshots, $\mathbf{G}$, where the time intervals between each graph are constant:

$$\mathbf{G} = \{\mathbf{G}_0, \mathbf{G}_1, \ldots, \mathbf{G}_T\}$$

.

As for one graph definition: $\mathbf{G}_t \in \mathbf{G}$ and $\mathbf{G}_t = \{\mathbf{V}_t, \mathbf{E}_t\}$ represents the graph at snapshot $t \in [0, T]$, where $\mathbf{V}_t, \mathbf{E}_t$ are the set of nodes and edges in $\mathbf{G}_t$, respectively.

In snapshot-based learning, link prediction typically focuses on the next immediate snapshot, rather than predicting further into the future as in event-based graphs.

2.2 Continuous Time Dynamic Graphs

CTDGs, also known as event-based methods, process temporal graphs as a stream of timestamped edge events. Edges in the graph $\mathbf{G}$ are represented as tuples where s_i and d_i are the source and target nodes, respectively, and t_i^s and t_i^e are the beginning and end times of the edge. Each edge is indexed by $i \in [0, k]$:

$$\mathbf{G} = \{(s_0, d_0, t_0^s, t_0^e), (s_1, d_1, t_1^s, t_1^e), \ldots, (s_k, d_k, t_k^s, t_k^e)\}$$

Compared to DTDGs, CTDGs are more flexible and effective by directly learning on the whole dynamic graph. Furthermore, node-based models first leverage the node information, such as temporal neighborhood or previous node history, to generate node embeddings and then aggregate node embeddings from both the source and destination node of an edge to predict its existence.

3 Related Work

3.1 UTG (Unifying Temporal Graph Network)

[3] proposed a unified framework that bridges the gap between snapshot-based and event-based temporal graphs. They developed a mapper that converts any discrete or continuous data into one of the two temporal formats, and then maps it back to the corresponding task. In discrete tasks, the prediction involves determining which edges will appear in a future snapshot, while in continuous tasks, the prediction is made for a future UNIX timestamp.

Given that traditional training processes for snapshot-based models are not well-suited for link prediction, [3] modified the standard training framework. This adaptation involves a streaming

framework where, during training, only inputs up to $\mathbf{G_{t-1}}$ are used to predict links in the current structure $\mathbf{G_t}$. Moreover, they set the backpropagation to occur at each snapshot, which is typical for event-based graphs.

3.2 DyGFormer

DyGFormer [6] is a transformer-based architecture for dynamic graph learning. It effectively addresses the limitation of previous methods, which often fail to capture both correlations among nodes and the long-term temporal dependencies inherent in dynamic graphs. Specifically, only historical first-hop interactions (u, v, t) of both source node u and destination node v before timestamp t are extracted firstly, which can be represented as $S_u^t = \{(u, u', t') \mid t' < t\} \cup \{(u', u, t') \mid t' < t\}$ and $S_v^t = \{(v, v', t') \mid t' < t\} \cup \{(v', v, t') \mid t' < t\}$ respectively. Instead of computing the encodings of neighbors, links, and time intervals for each sequence seperately, DyGFormer encodes the correlation between them as well by encoding the frequencies of every neighbor's appearances. The resulting four encoding sequences for u/v will then be patched to feed into the transformer for capturing long-term temporal dependencies. It is demonstrated that DyGFormer consistently achieves state-of-the-art results for link prediction task, which confirms the importance of modeling both node correlation and long-term temporal dependencies, which have been underestimated by previous works.

3.3 NAT (Neighborhood Aware Temporal Network)

In temporal graphs, tasks like link prediction rely heavily on the structure of joint neighborhoods [7]. Standard GNNs may produce identical embeddings for nodes with distinct neighborhoods, failing to capture this structure. NAT overcomes this by using neighborhood cache representations, dictionaries storing k-hop historical neighbors and their features, offering a richer, joint view of the neighborhood (see Figure 1 in [7]). The neighborhood representation for each node is defined as:

$$(Z_u^{(0)}(t), Z_u^{(1)}(t), \ldots, Z_u^{(K)}(t)),$$

where $Z_u^{(k)}$ denotes the $k - hop$ neighborhood representation, for $k = 0, 1, ..., K$.

Each time a new link is created between nodes u and node v, it updates the values of $Z_{u,v}^{(1)}$ using self representation of v: $Z_v^{(0)}$ and the previous interactions between the nodes: $Z_{u,v}^{(1)}$. This update employs learnable temporal methods, encoding the new timestamp t_i with Fourier features and learnable parameter, and then passing it through an RNN to update the connection. Using these representations, NAT identifies unique common nodes a of the current queried link between u and v, aggregates their representations across different k-neighborhoods, and applies several MLPs for future link predictions.

4 Method

4.1 Joint Neighborhood Features Encoding

4.1.1 DyGFormer Integration Methods

In this section, we explore how to integrate DyGFormer's [8] joint neighborhood representation as positional encodings into GCLSTM framework. DyGFormer integration methods consists of the following four key components.

1. **Extraction of Historical Interaction Sequences -** Given a source node u and a destination node v at timestep t, we first extract their historical first-hop interaction sequences respectively. Specifically, as illustrated in 3.2, we collect all interactions involving their immediate neighborhood prior to time t, resulting in two sequences S_u^t and S_v^t. Time step t is discretized following the quantization scheme used in GCLSTM. For sequences with varying lengths, we apply padding to standardize them to a uniform length.
2. **Neighborhood Feature Encoding -** Sequences extracted from previous step serve as the temporal neighborhoods where features will be extracted. For each historical sequence, we encode the involved neighbors based on the given features, resulting in $\mathbf{X}_{u,N}^t \in \mathbb{R}^{|S_u^t| \times d_N}$ and $\mathbf{X}_{v,N}^t \in \mathbb{R}^{|S_v^t| \times d_N}$. These features provides essential local structure information.

3. **Time Intervals Feature Encoding -** DyGFormer also leverages the time interval features to learn the periodic temporal patterns. Given the time interval $\Delta t' = t - t'$, the time interval feature $\mathbf{X}^t_{*,T} \in \mathbb{R}^{|S^t_u| \times d_T}$ is encoded as $\sqrt{\frac{1}{d_T}}[\cos(w_1\Delta t'), \sin(w_1\Delta t'), \cdots, \cos(w_{d_T}\Delta t'), \sin(w_{d_T}\Delta t')]$, where d_T is the feature dimension and w is learnable. By encoding the relative time intervals between each historical interaction and the current time t, the temporal patterns of node interactions are captured.
4. **Neighbor Co-occurrence Encoding -** To further exploit the correlation between the source node u and the destination node v , we additionally model the neighbor co-occurrence by encoding the frequency of each neighbor's appearance within the respective interaction sequence. Specifically, we compute the occurrence count of each neighbor and use it as an additional feature, resulting in the frequency encodings $\mathbf{C}^t_u \in \mathbb{R}^{|S^t_u| \times 2}$, and $\mathbf{C}^t_v \in \mathbb{R}^{|S^t_v| \times 2}$.

4.1.2 NAT Integration Methods

In this section, we explore how to integrate NAT's neighborhood representation. To this end, several key factors were considered:

1. **Pre-trained model -** As discussed in Section 3.3, NAT updates N-caches during link prediction training, maintaining up-to-date neighborhood representations while learning optimal weights. Since NAT stores these caches, we can pre-train the model on our data and, by freezing the weights, use the learned neighborhood representations as additional input to the UTG framework alongside the GCLSTM model.
2. **Normalization -** We assessed whether to apply normalization to the extracted NAT values before their integration into the link prediction model.
3. **Neighborhood representation -** We assessed three steps in the creation of NAT representation for integration into the UTG framework:

   (a) *Direct N-cache aggregation:* A straightforward method is to take the representation $Z^{(k)}_{u,v}$ for each link (u, v) and aggregate (we chose to use mean) the representations across the $k = 0, 1, \ldots, K$ neighborhoods. This method does not consider the intermediate or common neighbors $a \in A$ that u and v might share, and instead only leverages the learned representation of the specific queried link. Importantly, to use this method correctly, we first need to apply the hash function (*ncache_hash*) to retrieve the correct entry for each (u, v) pair. Then, we collect only the final dimensions (*self_dim* which is set to 4) of the *neighborhood_store*, which corresponds to the actual self-representation of the target node v in relation to u. The first three dimensions encode metadata: the last incoming node, the edge index, and the timestamp, and thus do not represent the historical context of the link.

   (b) *Joint neighborhood with Distance Encoding (DE):* A second, more structured method leverages NAT's joint neighborhood modeling. As described in [7], NAT first identifies all nodes $a \in A$ that appear in the joint K-hop N-caches of u and v at time t:

   $$a \in \left[\bigcup_{k=0}^{K} \text{key}(Z^{(k)}_u)\right] \cup \left[\bigcup_{k'=0}^{K} \text{key}(Z^{(k')}_v)\right]$$

   Each node a is associated with a binary DE vector $\text{DE}^t_{uv}(a) \in \mathbb{R}^{2K}$, defined as a concatenation of indicator vectors for a's membership in each hop's neighborhood of u and v, respectively. For example, if a is in both $Z^{(1)}_u$ and $Z^{(1)}_v$ at $t = 3$, then: $\text{DE}^{t_3}_{uv}(a) = [0, 1, 0] \oplus [0, 1, 0]$. NAT then concatenates DE with the aggregated representations of a across all hops $Z^{(k)}_{u,v,a}$, capturing its contextual role relative to both u and v (see Equations 1-2 in [7]).

   (c) *End-to-end MLP with attention:* A third option is to directly use the final contextualized neighborhood representation generated by NAT after applying multiple MLP layers and an attention mechanism over the joint neighborhood calculated in option *(b)*.

In order to explore the mentioned integration options, several adjustments were necessary. First, positive and negative samples during evaluation had to be separated. This was required due to the way NAT learns joint neighborhood representations, specifically, it only updates the N-caches based

on true connections. This separation is particularly important for option *(c)* from Step 3, where two distinct gradient backpropagation streams are applied for positive and negative links.

To ensure that this separation did not introduce bias into the results, we conducted two validation checks. First, we confirmed that baseline performance remained consistent, which it did. Second, we evaluated the impact of normalization on the distribution of NAT's embedding values. In each training step, one positive sample is contrasted with 999 negative samples, and applying normalization over such imbalanced batches could potentially skew the representation. However, we observed no significant difference in the mean or variance of NAT's embedding values between positive and negative samples, suggesting that normalization did not introduce bias and influenced the results.

4.2 Integration with GCLSTM for Link Prediction Task

1. *Baseline integration:* We defined the components involved in the model. Let $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^d$ represent the hidden states of the source and target nodes, respectively, as obtained from the GCLSTM. Let $\mathbf{h}_{uv} \in \mathbb{R}^d$ denote the joint neighborhood representation derived in the previous Section 4.1. The functions ϕ_1 and ϕ_2 are neural network layers applied to $\mathbf{h}_{uv}$, the first followed by a ReLU activation. The function f_{MLP} denotes a multi-layer perceptron applied to the element-wise interaction between $\mathbf{x}_i$ and $\mathbf{x}_j$. The final activation function σ is a sigmoid used to produce the prediction score.

   The model computes the link prediction score as follows:

$$\mathbf{x} = \mathbf{x}_i \odot \mathbf{x}_j \tag{1}$$

$$\mathbf{h}_{\text{trans}} = \phi_2\left(\text{ReLU}\left(\phi_1(\mathbf{h}_{uv})\right)\right) \tag{2}$$

$$\mathbf{z} = f_{\text{MLP}}(\mathbf{x}) \tag{3}$$

$$\hat{y}_{ij} = \sigma\left(\mathbf{W} \cdot [\mathbf{z} \,\|\, \mathbf{h}_{\text{trans}}] + b\right) \tag{4}$$

   Here, $\odot$ denotes element-wise multiplication, and $[\cdot \,\|\, \cdot]$ represents vector concatenation. The output $\hat{y}_{ij} \in (0, 1)$ is the predicted probability of a link between nodes i and j. Note that the only modifications made to the baseline architecture are in the second step and the concatenation step in the final layer. Regarding NAT, the dimensionality d of $\mathbf{h}_{uv}$ varies depending on which of the three integration options is used from Step 3. Specifically, in option *(a)*, $d = 4$; in option *(b)*, $d = \text{node\_dim} + 4 + \text{pos\_dim}$; and in option *(c)*, $d = \text{node\_dim} + 2 \times \text{self\_dim}$.

2. *Positional Encoding integration:* In a second variant, we replace the first transformation layer $\phi_1(\mathbf{h}_{uv})$ with a simple positional encoding function applied directly to $\mathbf{h}_{uv}$. This approach explores whether explicit structural positioning can serve as a more effective alternative to learned transformations in capturing neighborhood context.

5 Experiments

5.1 Experimental Settings

Datasets - We use the `tgbl-wiki` and `tgbl-review` datasets from the TGB benchmark [9], where each dataset is chronologically split into training (70%), validation (15%), and test (15%) sets based on edge timestamps.

Baselines - We compare our method with the chosen snapshot-based method GCLSTM.

Evaluation Metrics - We follow [3] to evaluate model performance for dynamic link prediction, which is treated as a ranking problem. Therefore, the Mean Reciprocal Rank (MRR) metric is applied. This metric calculates the reciprocal rank of the true destination node among a large number of possible destinations.

Model Configuration / Implementation - 999 negative edges are randomly sampled for each positive edge in the test set.

5.2 Finding DyGFormer's Best Configuration

5.2.1 Ablation of Different Encoding Scheme

To better understand the effectiveness of each neighborhood related features as discussed in our proposed encoding scheme, we conducted a series ablation experiments on the `tgbl-wiki` dataset.

As shown in Table 1, we progressively incorporate different types of neighborhood-based features into the baseline GCLSTM model.

It is observed that the MRR performance is improved to 0.405 by only incorporating the neighborhood features (NF), demonstrating the importance of capturing structural information from nodes' neighborhood for dynamic graph learning. Further integrating the time interval features (TF), the MRR gain is obtained to 0.423. It indicates that the temporal patterns of interactions plays a important role in dynamic graphs. Moreover, by encoding neighbor feature with neighborhood co-occurrence features (CF) leads to a larger improvement (MRR=0.448). It indicates that the correlation between source node and destination nodes is crucial for enhancing link prediction. When combining all three different neighborhood-based features (NF + TF + CF), the model achieves the highest performance (MRR=0.509) while remaining similar runtime for both training and evaluation, confirming the complementary benefits of neighborhood structure, temporal information and correlation between source neighborhood and destination neighborhood.

Table 1: Comparison of model performance and runtimes on the `tgbl-wiki` dataset with different encoding scheme using DyGFormer integration method.

Method	Performance	Training Runtime	Evaluation Runtime
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + NF	0.405 ± 0.018	16.21	3489.49
GCLSTM + NF + TF	0.423 ± 0.008	16.26	3457.45
GCLSTM + NF + CF	0.448 ± 0.027	16.23	3450.31
GCLSTM + NF + TF + CF	0.509 ± 0.016	16.24	3460.36

5.2.2 Ablation of Different Neighborhood Size

We further investigate the impact of the maximal number of neighbors sampled for each node when constructing the joint neighborhood features. Specifically, we vary the maximum number of neighbors from 8 to 64 and conduct experiments on `tgbl-wiki` dataset. As shown in Table 2, we observe that increasing the maximal number of neighbors sampled consistently improves the model's performance. Specifically, when increasing the maximum number of neighbors from 16 to 64, the MRR rises from 0.495 to 0.509. The possible reason is that when increasing the neighbors size, richer information of neighbors can be captured, such as more structural information. While increasing the neighbors size, the runtime for both training and evalution doesn't varies a lot, suggest that the DyGFormer integration method is able to scale without much computational costs.

Table 2: Comparison of model performance and runtimes on the `tgbl-wiki` dataset with different neighborhood size using DyGFormer integration method..

Method	Performance	Training Runtime	Evaluation Runtime
8	0.498 ± 0.011	16.22	3418.84
16	0.495 ± 0.003	16.25	3471.14
32	0.505 ± 0.009	16.22	3498.44
64	0.509 ± 0.016	16.24	3460.36

5.2.3 Ablation of Different Integration to Link Prediction Models

We also conducted experiments to investigate the impact of different integration strategies within the GCLSTM framework. As presented in Table 3, we compare two different methods as explained in Section 4.2. It is observed that the baseline integration achieves a significant better MRR result of 0.509, comparing to the GCLSTM baseline with MRR of 0.361. On the other hand, using joint neighborhood features as the positional encoding leads to a noticeable performance to 0.400, indicating that leveraging neighborhood information actually helps the dynamic graph learning. This lower performance comparing to integration baseline highlights that using neigborhood information solely as positional encoding may add additional complexity to learn and make optimization more difficult. This observation is also supported by the longer training runtime of the positional encoding strategy, reflecting its increased computational burden.

Table 3: Comparison of model performance and runtimes between two predictive models on the `tgbl-wiki` dataset using DyGFormer integration method.

Method	Performance	Training Runtime	Evaluation Runtime
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + DyGFormer (baseline integration)	0.509 ± 0.016	16.24	3460.36
GCLSTM + DyGFormer (positional encoder)	0.400 ± 0.005	23.20	3889.33

5.3 Finding NAT's Best Configuration

We used the default hyperparameters: a 2-hop neighborhood with sizes 16 and 32, and a hidden layer size of 256 for both GCLSTM and link prediction models. Hyperparameter tuning was conducted on the `tgbl-wiki` dataset. Due to computational constraints, training was limited to 200 epochs, with evaluations every 50 epochs across three random seeds.

The best results were achieved using a pre-trained NAT model with output normalization, neighborhood representation from integration option *(b)*, and the baseline integration method. These settings - corresponding to Steps 1, 2, 3, and Section 4.2 - are used as defaults throughout this section and are only modified when explicitly evaluated.

5.3.1 Pre-trained Model

We compared two integration approaches: (1) jointly training NAT with the GCLSTM model, and (2) using a pre-trained NAT model.

As shown in Table 4, both approaches yielded similar MRR performance, with a slight edge for the pre-trained setup. While we expected pretraining to reduce training time by avoiding NAT gradient updates, this was not evident, possibly because the NAT module itself does not involve particularly complex backpropagation operations. Moreover, since the experiments were conducted on rented cloud infrastructure, variability in GPU resource availability and potential background processes may have influenced runtime measurements, making them less reliable for direct comparison. Nevertheless, we chose to use the pre-trained NAT configuration for all subsequent experiments.

Table 4: Comparison of model performance and runtimes between training NAT jointly and using a pre-trained NAT model on the `tgbl-wiki` dataset.

Method	Performance	Training Runtime	Evaluation Runtime
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + NAT (pretrained)	0.565 ± 0.006	12.19	181.21
GCLSTM + NAT (jointly trained)	0.559 ± 0.017	12.28	184.12

5.3.2 Normalization

Table 5 shows the results of applying min-max normalization to the NAT output embeddings, scaling values to the range $[0, 1]$. Without normalization, MRR scores dropped below those of the GCLSTM baseline, underscoring its importance. We hypothesize that normalization contributes to greater training stability, making it easier for the downstream model to learn from the neighborhood representations.

5.3.3 Neighborhood Representation

Table 6 compares the three neighborhood representation strategies for NAT integration. Option *(a)* yields the lowest MRR but offers the fastest runtime, likely because it relies solely on the individual N-caches of source and target nodes, without joint neighborhood computation.

Interestingly, it underperforms even the GCLSTM baseline. This may be due to redundancy or misalignment between the NAT-derived features and the GCLSTM embeddings, which are generated

Table 5: Comparison of model performance and runtimes between normalizing and not normalizing NAT's output on the `tgbl-wiki` dataset.

Method	Performance	Training Runtime	Evaluation Runtime
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + NAT (normalized)	0.565 ± 0.006	12.19	181.21
GCLSTM + NAT (unnormalized)	0.350 ± 0.006	11.91	183.47

through different mechanisms. The conflicting representations may confuse the downstream model, reducing accuracy. Although a transformation layer could help reconcile these differences, the current implementation appears insufficient, suggesting that more expressive architectures may be needed to effectively integrate option *(a)*.

Option *(c)*, on the other hand, improves over the baseline by incorporating contextualized neighborhood information via NAT's architecture, but its runtime approaches that of training NAT directly and still falls short of NAT's standalone performance. In contrast, option *(b)* offers a more balanced trade-off, outperforming both option *(a)* and the GCLSTM baseline, while running faster than option *(c)*. It leverages joint neighborhood representations without the added cost of attention layers and prediction heads.

Table 6: Comparison of model performance and runtimes between three output options of NAT on the `tgbl-wiki` dataset. An asterisk (*) indicates results sourced from [9] and not generated locally.

Method	Performance	Training Runtime	Evaluation Runtime
*Only NAT (not using UTG)	0.749 ± 0.010	-	340.51
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + NAT (option *(a)*)	0.334 ± 0.005	7.93	58.6
GCLSTM + NAT (option *(b)*)	0.565 ± 0.006	12.19	181.21
GCLSTM + NAT (option *(c)*)	0.523 ± 0.018	13.87	238.86

5.3.4 Link Prediction Models

We tested several strategies for integrating NAT-derived features into the link prediction model, as described in Section 4.2.

As shown in Table 7, this method did not improve performance over the GCLSTM baseline, unlike the DyGFormer representation (see Section 5.2.3). We hypothesize that this is because NAT already encodes temporal position through its edge-time-aware updates (see Section 3.3), making additional Fourier-based encoding redundant, whereas DyGFormer does not use any positional encoding. This further supports the hypothesis from the previous section, that integrating redundant information may have no effect or even degrade model performance.

Nonetheless, this observation opens up future directions, such as exploring alternative encodings like Reading Order Equivariant Positional Encoding (RoPE)[10], which may offer more effective task-specific representations (further discussed in Section6).

Table 7: Comparison of model performance and runtimes between two predictive models on the `tgbl-wiki` dataset.

Method	Performance	Training Runtime	Evaluation Runtime
Only GCLSTM	0.361 ± 0.008	6.70	24.12
GCLSTM + NAT (baseline integration)	0.565 ± 0.006	12.92	181.21
GCLSTM + NAT (positional encoder)	0.375 ± 0.008	12.16	186.74

5.3.5 Comparing Datasets and Neighborhood Size

After identifying the optimal configuration for `tgbl-wiki` dataset, we evaluated the impact of neighborhood size on the larger `tgbl-review` dataset (Table 8).

Due to the higher computational cost of computing joint neighborhoods in `tgbl-review` dataset, applying the same settings as `tgbl-wiki` dataset caused out-of-memory errors (see Appendix A). We resolved this by reducing the hidden size and batch size to 64. We also found that normalization introduced bias in `tgbl-review` dataset, causing a sharp drop in MRR from 0.36-0.47 in epoch 0 to as low as 0.06. This was due to differing variance in positive versus negative samples; therefore, we disabled normalization for this dataset.

In `tgbl-wiki` dataset, all hop sizes performed similarly, with 1-hop neighborhoods of size 32 and 64 slightly outperforming others. However, size 64 had higher evaluation time, making size 32 the best trade-off between accuracy and efficiency. By contrast, our method did not outperform the baseline on `tgbl-review` dataset, and performance was consistent across hop sizes. Due to the high computational cost, we could not investigate the cause of this behavior further. However, one possible reason is that the reduced embedding size may be insufficient to capture the more complex dynamics of this larger dataset.

Additionally, Figure 1 illustrates the training loss and MRR scores over time. The training process for `tgbl-review` dataset appears to be less stable compared to `tgbl-wiki` dataset, potentially contributing to the observed performance limitations.

Table 8: Comparison of performance across two datasets with different neighborhood sizes and their respective evaluation runtimes. All assessments, except for those noted in the first and second rows, were conducted using the selected GCLSTM+NAT model. An asterisk (*) indicates results sourced from [9] and not generated locally.

Size/Model	tgbl-wiki	tgbl-wiki Eval. Runtime	tgbl-review	tgbl-review Eval. Runtime
* only NAT	0.749 ± 0.010	340.51	0.341 ± 0.020	8925.21
only GCLSTM	0.361 ± 0.008	24.12	0.063 ± 0.000	522.05
8	0.565 ± 0.004	131.43	0.064 ± 0.000	5214
16	0.564 ± 0.027	138.73	0.062 ± 0.001	5212.73
32	0.575 ± 0.005	168.90	0.063 ± 0.001	5101.84
64	0.579 ± 0.011	230.93	0.063 ± 0.000	5183.92
16,32	0.565 ± 0.006	181.21	0.064 ± 0.001	5851.05

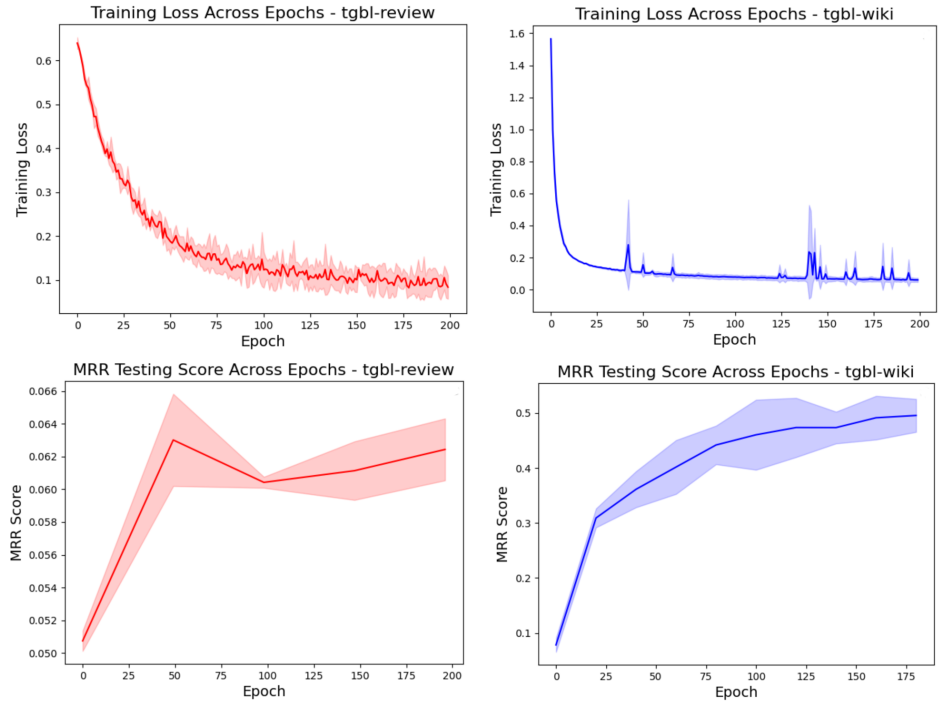

Figure 1: Training and evaluation results across epochs for both `tgbl-wiki` and `tgbl-review` datasets. The training loss (top row) and MRR testing score (bottom row) are shown for the selected model using a 2-hop neighborhood with sizes 16 and 32. Each curve represents the average performance across three random seeds (1, 2, and 3), with the shaded areas indicating the standard deviation.

5.4 Contribution of Joint Neighborhood Features

Based on ablation studies from previous sections, we selected the best-performing configurations by integrating DyGFormer-based and NAT-based joint neigborhood features into GCLSTM framework. Table 9 clearly shows the improvements compared to baselines.

Table 9: Best performance across two datasets. All assessments, except for those noted in the first, second and third rows, were conducted using the selected GCLSTM+NAT model. An asterisk (*) indicates results sourced from [9] and not generated locally.

Size/Model	tgbl-wiki	tgbl-wiki Eval. Runtime	tgbl-review	tgbl-review Eval. Runtime
* NAT	0.749 ± 0.010	340.51	0.341 ± 0.020	8925.21
* DyGFormer	0.798 ± 0.004	7196.52	0.224 ± 0.015	26477.51
GCLSTM	0.361 ± 0.008	24.12	0.063 ± 0.000	522.05
GCLSTM + DyGFormer	0.509 ± 0.016	3460.36	0.072 ± 0.002	18376.92
GCLSTM + NAT	0.579 ± 0.011	230.93	0.064 ± 0.001	5183.92

6 Conclusion & Future Work

In this project, we propose an effective approach to enhance snap-shot based methods for dynamic graph learning, by incorporating joint neighborhood features. Specifically, we leverages DyGFormer-based joint and NAT-based joint neighborhood features into the GCLSTM framework.

Both new models outperformed the baseline on `tgbl-wiki` dataset, suggesting that incorporating neighborhood features plays a key role in link prediction and can further enhance snapshot-based models under UTG framework. However, the models did not outperform the event-based approaches, likely because the latter use finer-grained data. Unlike UTG, which aggregates multiple links under

the same timestamp when converting from CTDG to DTDG, event-based models preserve individual link events.

Interestingly, the models did not outperform the `tgbl-review` baseline obtained on our machine, which is lower than reported in [2] due to testing every 50 epochs instead of selecting the best validation checkpoint. This may also be attributed to the reduced batch and hidden size. Moreover, interactions in this dataset are sparser, which can make neighborhood features less informative. As a result, incorporating joint features might not yield a significant improvement. However, further evaluation is needed on this dataset.

When compared our two methods, NAT had a higher MRR score and ran faster than DyGFormer. While our models show promising results, several directions for further investigation could be explored.

For DyGFormer, even though neighborhood co-occurrence by frequency has demonstrated significant effectiveness, it can be further improved using more flexible techniques, such as importance encoding by attention mechanism. On the other hand, multi-hop historical interactions can potentially enrich the temporal and structural information, which is helpful for link prediction. Moreover, more advanced encoding strategies such as feature-wise attention could be explored to make it more efficient, considering the current notable runtime overhead.

For NAT, future work could explore alternative positional encodings to better capture complex temporal or structural patterns. For example, RoPE, initially designed for spatial reasoning in document graphs [10], might capture aspects of the data not yet fully leveraged with the Fourier-based encoding. Further research could explore alternative aggregation techniques for neighbourhood representation per link and the promising co-occurrence approach used in the DyGFormer model.

Acknowledgements

We would like to thank Dr. Andy Huang for his guidance during the project. We would also like to thank Dr. Petar Velickovic and Prof. Pietro Liò for delivering the lectures and designing the L65 course.

References

[1] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70):1–73, 2020. 1

[2] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems*, 36, 2024. 1, 11

[3] Shenyang Huang, Farimah Poursafaei, Reihaneh Rabbany, Guillaume Rabusseau, and Emanuele Rossi. Utg: Towards a unified view of snapshot and event based models for temporal graphs. *arXiv preprint arXiv:2407.12269*, 2024. 2, 5

[4] Kaike Zhang, Qi Cao, Gaolin Fang, Bingbing Xu, Hongjian Zou, Huawei Shen, and Xueqi Cheng. Dyted: Disentangled representation learning for discrete-time dynamic graph. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3309–3320, 2023. 2

[5] Ming Jin, Yuan-Fang Li, and Shirui Pan. Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs. *Advances in Neural Information Processing Systems*, 35:19874–19886, 2022. 2

[6] Chao Yang, Xianzhi Wang, Lina Yao, Guodong Long, and Guandong Xu. Dyformer: A dynamic transformer-based architecture for multivariate time series classification. *Information Sciences*, 656:119881, 2024. 3

[7] Yuhong Luo and Pan Li. Neighborhood-aware scalable temporal network representation learning. In *Learning on Graphs Conference*, pages 1–1. PMLR, 2022. 3, 4

[8] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems*, 36: 67686–67700, 2023. 3

[9] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems*, 36:2056–2073, 2023. 5, 8, 9, 10

[10] Chen-Yu Lee, Chun-Liang Li, Chu Wang, Renshen Wang, Yasuhisa Fujii, Siyang Qin, Ashok Popat, and Tomas Pfister. Rope: reading order equivariant positional encoding for graph-based document information extraction. *arXiv preprint arXiv:2106.10786*, 2021. 8, 11

A Computing Resources

NAT experiments were conducted using cloud GPU instances provided by Vast.ai. The software environment was based on the official `vastai/pytorch:2.5.1-cuda-12.1.1` Docker image, which includes PyTorch 2.5.1 and CUDA 12.1.1. The selected machine for the `tgbl-wiki` dataset was equipped with four NVIDIA RTX A4000 GPUs, each with 16 GB of VRAM, and a total of 64 GB VRAM. The selected machine for the `tgbl-review` dataset was equipped with a single NVIDIA A100 SXM4 GPU, offering 80 GB of VRAM.

DyGFormer experiments were conducted using local GPU machine and online servers provided by AutoDL platform. The software environment was configured based on Pytorch 2.5.1 with CUDA 11.8. For the `tgbl-wiki` dataset, the experiments were performed on a local machine equipped with a single 4090 GPU with 24 GB of VRAM. For the `tgbl-review` dataset, the experiments were conducted on a machine equipped with a single NVIDIA V100 GPU, offering 32 GB of VRAM.